

**MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY**

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Final Group Project Report submitted for BDA501 – Big Data

**MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY**

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

by

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Final Group Project Report submitted for BDA501 – Big Data

Project repository: <https://github.com/RocketUp-Team/Fraud-Detection-Score-Risk>

**Supervisor:
TAN Le Duy**

Fraud Detection and Risk Scoring System Using Apache Spark and Machine Learning

RocketUpTeam

AN Duong Binh	TUYEN Le Quang
LONG Pham Duc	TRUNG Do Quoc
QUAN Duong Hong	NHI Nguyen Le Hong

Degree Master of Software Engineering

FPT University

2026

Abstract

This thesis presents an academic fraud risk scoring workflow for the IEEE-CIS dataset. Apache Spark performs typed ingestion, transaction–identity joining, quality auditing, chronological leakage-controlled processing, feature engineering, and Parquet handover. Python model training compares Logistic Regression, LightGBM, XGBoost, and CatBoost after the Spark-to-pandas boundary. The system packages feature mappings, calibration, thresholds, and explanations for an in-process FastAPI service with a React review interface.

The current CatBoost balanced candidate reports validation ROC-AUC 0.9081210 and PR-AUC 0.5805993, with holdout ROC-AUC 0.8780755 and PR-AUC 0.4266435. The current CatBoost candidate is labelled `v2` in its metadata, but its promotion decision is `not_promoted`; the configured V2 software default remains unchanged. The principal limitations are offline–online historical-feature mismatch, duplicated policy ownership, local lineage, single-node training, fail-open fallback, and incomplete security and monitoring controls. The result is a bounded academic demonstration, not a production-readiness claim.

ACKNOWLEDGMENTS

The members of RocketUpTeam sincerely thank supervisor TAN Le Duy for guidance and feedback provided throughout project development and documentation.

Team Roster and Repository Role Mapping

Table 1 lists the project team members, official roles, and primary repository responsibilities.

Table 1: *Team Roster and Repository Role Mapping*

Member Name	Role	Primary Responsibility
TUYEN Le Quang	Project Leader	Responsible for system ideation and overall project management.
NHI Nguyen Le Hong	Content & Presentation Coordinator	Responsible for synthesizing project content, designing slides, and developing the presentation strategy.
AN Duong Binh	Big Data & Pipeline Engineer	Responsible for Spark data ingestion, left joins, temporal splitting, and feature contract verification.
LONG Pham Duc	Frontend & UI Engineer	Responsible for React dashboard UI, risk visualizations, transaction queue, and analyst review workflow.
TRUNG Do Quoc	Backend & API Engineer	Responsible for FastAPI scoring service, database persistence, and scoring contracts.
QUAN Duong Hong	Machine Learning Engineer	Responsible for classifier training, hyperparameter tuning, SHAP explainability, isotonic calibration, and hold-out evaluation.

AI Assistance and Verification Disclosure

In accordance with academic integrity standards, the project team discloses the responsible use of Artificial Intelligence (AI) tools during initial project planning, code drafting, and documentation formatting.

The team explicitly affirms that:

- **Human Ownership and Verification:** All AI-suggested code snippets, architectural ideas, and documentation drafts were thoroughly inspected, modified, refactored, and verified by team members.
- **Empirical Runtime Validation:** All Apache Spark ETL scripts, machine learning estimators, isotonic calibration models, FastAPI REST endpoints, and React UI components were independently executed, unit-tested, and verified via automated test suites.
- **Data Integrity:** All metrics, PR-AUC/ROC-AUC values, holdout evaluation results, checksums, and promotion gate evidence in this report are derived strictly from empirical runtime execution logs and verified data contracts.

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Full Definition / Meaning
API	Application Programming Interface
AUC	Area Under the Curve
BDA	Big Data Analytics
CSV	Comma-Separated Values
EDA	Exploratory Data Analysis
FPT	Corporation / FPT University
GBDT	Gradient Boosted Decision Trees
HTTP	Hypertext Transfer Protocol
IEEE-CIS	IEEE Computational Intelligence Society
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LR	Logistic Regression
ML	Machine Learning
MLlib	Machine Learning Library (Apache Spark)
PR-AUC	Precision-Recall Area Under the Curve
ROC-AUC	Receiver Operating Characteristic Area Under the Curve
SHAP	SHapley Additive exPlanations
SQL	Structured Query Language
UI	User Interface
V2	Version 2 Model Release Authority

CONTENTS

Abstract	3
Acknowledgments	4
List of Abbreviations and Acronyms	5
1 Introduction	10
1.1 Problem context	10
1.2 Objectives & research questions	10
1.3 BDA501 requirement compliance	10
2 Background and Related Work	11
2.1 Imbalanced temporal ranking	11
2.2 IEEE-CIS dataset structure	11
2.3 Technology boundary	11
3 Requirements and System Architecture	12
3.1 Functional requirements	12
3.2 Quality requirements	12
3.3 Integrated architecture	12
3.4 System contracts	13
3.5 Architectural tradeoffs	14
3.6 Main source code components	14
3.7 Distributed Spark execution evidence	14
4 Big-Data Processing and Feature Engineering	16
4.1 Source validation and typed ingestion	16
4.2 Join integrity	16
4.3 Leakage-controlled transformation order	16
4.4 Chronological split	16
4.5 Feature engineering & class balance	17
4.6 Verification result	17
5 Model Development, Evaluation, and Serving	18
5.1 Artifact identity and terminology	18
5.2 Model-development protocol	18
5.3 PySpark MLlib DecisionTree baseline	18
5.4 Candidate models and selection	19
5.5 Current CatBoost candidate results	19
5.6 Packaging, checksum, and promotion	19

5.7	Request-time scoring	20
6	Application and Deployment	21
6.1	Integrated serving architecture	21
6.2	Scoring and review workflow	21
6.3	Deployment boundary	21
7	Validation and Whole-System Review	22
7.1	Evidence status	22
7.2	Technical challenges and implemented solutions	23
7.3	Consolidated limitations	23
8	Conclusion and Future Work	24
8.1	Conclusion	24
8.2	Summary of research question answers	24
8.3	Future work	24
	References	25
A	API Reference	26
A.1	Common enums & core fields	26
A.2	Endpoint summary	26
B	Deployment and Feature Contracts	27
B.1	Canonical feature groups	27
C	Runbook and Role Mapping	28
C.1	Acceptance scenarios	28

LIST OF FIGURES

3.1	Overall system architecture	13
3.2	Data, feature, and evaluation contracts	13
4.1	Detailed data processing pipeline	16
5.1	Model training lifecycle	18
5.2	Promotion state machine	20
6.1	Application User Interface: Real-time risk scoring, inspection queue, and analyst review workflow.	21
7.1	Exploratory Data Analysis and Model-Family Selection Evidence.	22
7.2	Candidate holdout evaluation & confusion matrix.	22
7.3	Probability calibration: Raw vs isotonic-calibrated Brier score.	23
B.1	Default application deployment	27

LIST OF TABLES

1	Team Roster and Repository Role Mapping	4
1.1	BDA501 Requirement-Compliance Matrix	10
3.1	Functional requirements specification	12
3.2	Quality requirements specification	12
3.3	Major architectural tradeoffs	14
3.4	Main Source Files and Component Responsibilities	14
3.5	Distributed Spark Execution Evidence	15
4.1	Observed source files	16
4.2	Transaction–identity join audit	16
4.3	Chronological labeled splits	17
4.4	Training and evaluation class distributions	17
5.1	Model identifiers and runtime roles	18
5.2	PySpark MLlib DecisionTree Baseline Performance	18
5.3	Current model-family selection evidence	19
5.4	Current CatBoost candidate validation and holdout metrics	19
5.5	Current candidate promotion gate	19
7.1	Technical Challenges, Implemented Solutions, and Evidence	23
A.1	Core API fields	26
A.2	Core REST Endpoints	26
B.1	Canonical feature groups	27
C.1	Acceptance scenarios	28

CHAPTER 1

INTRODUCTION

1.1 Problem context

Online transaction fraud is an imbalanced classification problem with rare positive instances and asymmetric error costs [1, 2].

How can a reproducible big-data pipeline, an imbalanced-classification model, an explainable scoring API, and a human-review interface be integrated into one auditable fraud risk scoring system?

1.2 Objectives & research questions

- Ingest and validate the **1.29 GB** IEEE-CIS dataset via Apache Spark.
- Construct leakage-free chronological dataset splits (`train`, `val`, `holdout`).
- Compare MLlib DecisionTree, LightGBM, XGBoost, and CatBoost classifiers.
- Serve packaged models behind a FastAPI scoring API with TreeSHAP explanations and React UI.

RQ1: Can raw data be processed without identifier overlap or temporal leakage?

RQ2: Which classifier achieves highest validation PR-AUC and retains holdout performance?

RQ3: Can the model be served efficiently without a Spark dependency at request time?

RQ4: What technical boundaries separate the academic demo from production readiness?

1.3 BDA501 requirement compliance

Table 1.1 summarizes compliance with BDA501 project rubric requirements.

Table 1.1: *BDA501 Requirement-Compliance Matrix*

Requirement	Project evidence
Main tool: Apache Spark	PySpark DataFrame & MLlib pipeline in Spark Preprocessing Module
Dataset size $\geq$ 500 MB	4 raw IEEE-CIS CSV files totaling 1.29 GB in raw dataset store
Raw-data ingestion	Explicit Spark schemas & required file validation in ingestion pipeline
Cleaning & transform	Stateless cleanup, identity normalization, missingness handling
Descriptive analysis	Spark <code>groupBy</code> , <code>agg</code> , <code>Window</code> analytics & EDA reporting module
Machine learning	PySpark MLlib DecisionTree baseline & CatBoost/LightGBM/XGBoost comparison
Distributed cluster	2 -worker Docker Compose Spark standalone cluster configuration
Architecture diagram	End-to-end system architecture & contract diagrams in documentation
Installation instructions	Project <code>README</code> and Docker Compose full-stack orchestration
Main source code	Modular source tree (<code>pipeline</code> , <code>model</code> , <code>backend</code> , <code>frontend</code>)
Analysis results	Saved JSON/Parquet metrics, PR-AUC, confusion matrix, calibration reports
Challenges & solutions	Summarized in Section 1.4 & Technical Challenges Matrix
Demo readiness	Full-stack Docker Compose deployment with React UI & FastAPI scoring backend

CHAPTER 2

BACKGROUND AND RELATED WORK

2.1 Imbalanced temporal ranking

Fraud detection is framed as binary temporal ranking on rare positive cases (fraud rate $\approx 3.5\%$). Chronological splitting along `TransactionDT` preserves temporal order and prevents look-ahead leakage.

2.2 IEEE-CIS dataset structure

The dataset contains **590,540** training transactions and optional identity records [1]. Preserving transaction row authority via LEFT JOIN prevents data loss while capturing missingness indicators.

2.3 Technology boundary

Architecture operates via **distributed Spark data preparation with single-node Python estimator training**. CatBoost, LightGBM, XGBoost, and Logistic Regression are evaluated, using TreeSHAP for feature attribution [3].

CHAPTER 3

REQUIREMENTS AND SYSTEM ARCHITECTURE

3.1 Functional requirements

Table 3.1: *Functional requirements specification*

ID	Requirement Description
FR-01	Validate and process required IEEE-CIS transaction and identity CSV files.
FR-02	Produce train, validation, holdout, and unlabeled model-ready datasets.
FR-03	Preserve canonical feature order and preprocessing metadata.
FR-04	Compare binary classifiers using validation PR-AUC metrics.
FR-05	Score transactions and return probability, risk score, version, and explanations.
FR-06	Persist scored transactions and human review outcomes.
FR-07	Filter, sort, paginate, and inspect transaction records.
FR-08	Import CSV and load processed datasets with progress reporting.
FR-09	Support review approval/rejection and fraud/legitimate labels.
FR-10	Present risk, SHAP evidence, and model state in a browser interface.

3.2 Quality requirements

Table 3.2: *Quality requirements specification*

ID	Requirement Description
QR-01	Temporal leakage control and non-overlapping chronological splits.
QR-02	Reproducible schema and feature contracts.
QR-03	Version-aware model artifacts and rollback capability.
QR-04	Explainable tree-model predictions via TreeSHAP.
QR-05	Local containerized deployment with bounded data mounts.
QR-06	Automated subsystem test suite.
QR-07	Auditable experiment and promotion lineage.
QR-08	Durable background job operations.
QR-09	Role-based authentication, authorization, and audit logging.
QR-10	Real-time production latency and drift monitoring.

3.3 Integrated architecture

The system integrates batch data preparation, offline ML training, and online inference into an in-process modular monolith. Figure 3.1 presents the overall architecture and data flow contracts.

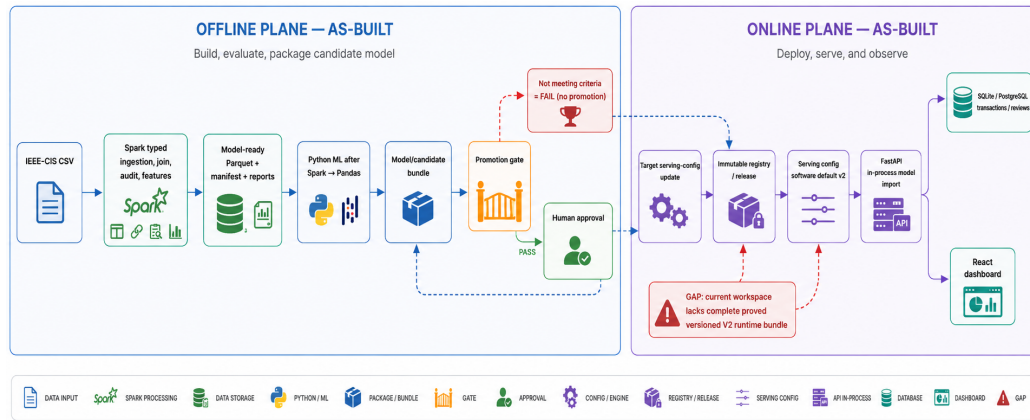


Figure 3.1: Overall system architecture.

3.4 System contracts

The system enforces three primary contracts:

1. **Data Contract:** Canonical 68-feature vector and schema hash.
2. **Scoring Contract:** Per-transaction request/response, probability, and TreeSHAP.
3. **API Contract:** Pydantic & TypeScript REST endpoints.

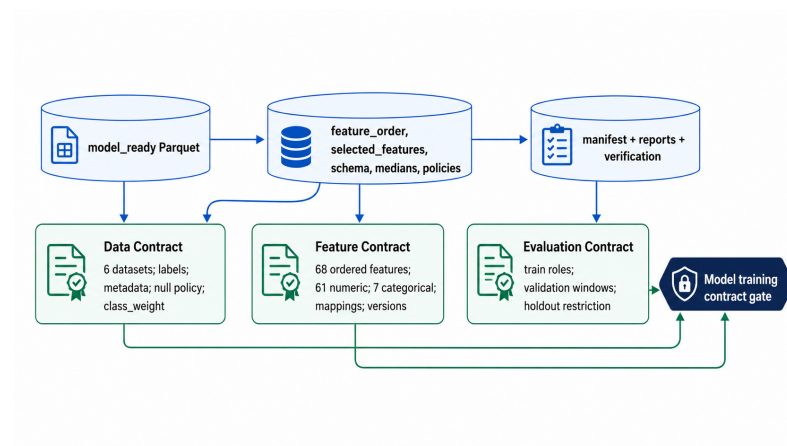


Figure 3.2: Data, feature, and evaluation contracts.

3.5 Architectural tradeoffs

Table 3.3: *Major architectural tradeoffs*

Choice	Benefit	Cost
Monorepo and path dependency	Simple handoff and one Compose stack	Backend inherits model dependencies
Spark then pandas	Scalable preparation and familiar estimators	Driver-memory training ceiling
Fixed backend risk bands	Stable UI and simple tests	Not tied to model operating evidence
In-memory jobs	Minimal infrastructure	Lost on restart and single-worker only
SQLite local default	Fast developer startup	Different operational behavior from PostgreSQL
Heuristic fallback	Demo continuity	Can conceal model loading failures
Manual TypeScript types	No generator setup	Contract drift risk

3.6 Main source code components

Table 3.4 outlines the primary source implementation files and core responsibilities across project components.

Table 3.4: *Main Source Files and Component Responsibilities*

Component	Module reference	Responsibility
Spark Preprocessing	Preprocessing Pipeline	Schema ingestion, left join, leakage-free transformation, Parquet export
Data Verifier	Verification Engine	Validates dataset row counts, schema hash, split overlap, verification report
Model Training	Estimator Training Module	Single-node candidate training, tuning, calibration, holdout evaluation
Backend Scoring	FastAPI Scoring Service	Artifact loading, probability scoring, SHAP calculation, FastAPI endpoints
Frontend Dashboard	React UI Dashboard	React UI, risk queue, SHAP feature rendering, analyst review workflow
Docker Deployment	Multi-container Orchestration	Multi-container service orchestration (backend, frontend, db, preprocessing)

3.7 Distributed Spark execution evidence

Table 3.5 presents the Spark execution configurations and deployment specification available in the repository.

Table 3.5: *Distributed Spark Execution Evidence*

Spark mode	Master URL	Wk	Cpu	Mem	Out	Deployment Spec
Local Preproc	<code>local[4]</code>	1	4	8GB	280MB	Preprocessing Service Spec
Cluster Mode	<code>spark://master:7077</code>	2	4	8GB	280MB	Standalone Cluster Spec

Note: Cluster configuration is implemented, but a clean end-to-end distributed runtime was not independently verified.

CHAPTER 4

BIG-DATA PROCESSING AND FEATURE ENGINEERING

4.1 Source validation and typed ingestion

The raw IEEE-CIS dataset contains **4** CSV files totaling **1.29 GB** (Table 4.1). The Spark pipeline applies explicit schemas and file discovery checks.

Table 4.1: *Observed source files*

File	Bytes	Role
train_transaction.csv	683,351,067	Labeled transactions
train_identity.csv	26,529,680	Optional train identity
test_transaction.csv	613,194,934	Unlabeled transactions
test_identity.csv	25,797,161	Optional test identity

4.2 Join integrity

Row authority is preserved via a LEFT JOIN on `TransactionID` (Table 4.2).

Table 4.2: *Transaction–identity join audit*

Dataset	Transactions	Identity matches	Unmatched	Row difference
Train	590,540	144,233	446,307	0
Test	506,691	141,907	364,784	0

4.3 Leakage-controlled transformation order

Transformations fit strictly on training partitions and apply forward to validation and holdout. Figure 4.1 details the execution pipeline.



Figure 4.1: *Detailed data processing pipeline.*

4.4 Chronological split

Sorting by `TransactionDT` creates non-overlapping chronological splits (Table 4.3).

Table 4.3: *Chronological labeled splits*

Dataset	Rows	Fraud	Fraud rate	TransactionDT range
Train	412,956	14,522	3.5166%	86,400–10,432,902
Validation	88,486	3,036	3.4310%	10,432,915–13,136,583
Holdout	89,098	3,105	3.4849%	13,136,627–15,811,131

4.5 Feature engineering & class balance

The canonical **68**-feature vector includes Time, Amount, Missingness, Anonymized behavior (C1--C14, D1--D15), Historical entity aggregates, and Categoricals.

Table 4.4: *Training and evaluation class distributions*

Dataset	Legitimate	Fraud	Ratio
train_original	398,434	14,522	27.44
train_weighted	398,434	14,522	27.44
train_balanced	43,872	14,522	3.02
validation	85,450	3,036	28.15
holdout	85,993	3,105	27.70

4.6 Verification result

The automated verifier passes **94** checks with zero failures, confirming handoff readiness:

```
ready_for_downstream_training
```

CHAPTER 5

MODEL DEVELOPMENT, EVALUATION, AND SERVING

5.1 Artifact identity and terminology

The project evaluates four model states: Historical V2 (LightGBM), Serving Default V2 (Configured champion), and Current Candidate (CatBoost Balanced, status: `not_promoted`) (Table 5.1).

Table 5.1: *Model identifiers and runtime roles*

Identifier	Model	Role	Status
Historical V2	LightGBM	Historical artifact	Historical
Serving default v2	LightGBM	Configured champion	Unchanged
Current candidate	CatBoost balanced	Challenger	Not promoted

5.2 Model-development protocol

Architecture operates via **distributed Spark data preparation with single-node Python estimator training**. Figure 5.1 illustrates the end-to-end training lifecycle.



Figure 5.1: *End-to-end model training lifecycle.*

5.3 PySpark MLlib DecisionTree baseline

To satisfy BDA501 big-data ML baseline requirements, a PySpark MLlib `DecisionTreeClassifier` pipeline evaluates the **68**-feature vector across three dataset variants (Table 5.2).

Table 5.2: *PySpark MLlib DecisionTree Baseline Performance*

Model variant	Split	PR-AUC	ROC-AUC	Precision	Recall	Time (s)
baseline_tree	Validation	0.0205	0.2329	0.6928	0.1574	13.5
baseline_tree	Holdout	0.0209	0.2366	0.6641	0.1948	13.5
weighted_tree	Validation	0.2666	0.6866	0.1564	0.6400	12.1
weighted_tree	Holdout	0.2773	0.6984	0.1494	0.6583	12.1
undersampled_tree	Validation	0.0284	0.4265	0.2791	0.5217	5.7
undersampled_tree	Holdout	0.0278	0.4163	0.2039	0.5076	5.7

5.4 Candidate models and selection

Evaluation compares Logistic Regression, LightGBM, XGBoost, and CatBoost (Table 5.3). CatBoost Balanced achieved highest PR-AUC.

Table 5.3: *Current model-family selection evidence*

Model	Training variant	ROC-AUC	PR-AUC	Decision
Logistic Regression	Weighted	0.8219	0.3090	Baseline
LightGBM	Weighted	0.9116	0.5682	Compared
XGBoost	Weighted	0.8879	0.5529	Compared
CatBoost	Weighted	0.8757	0.5245	Compared
CatBoost	Balanced	0.9070	0.5703	Selected

5.5 Current CatBoost candidate results

Table 5.4 records candidate performance across validation and holdout splits.

Table 5.4: *Current CatBoost candidate validation and holdout metrics*

Window	Threshold	PR-AUC	ROC-AUC	Precision	Recall	Brier
Validation selection	0.50	0.5806	0.9081	0.3970	0.6220	–
Holdout	0.16	0.4266	0.8781	0.3820	0.5182	0.0248

5.6 Packaging, checksum, and promotion

The candidate promotion gate evaluated 9 criteria (Table 5.5). Due to holdout PR-AUC degradation (**0.4266** vs target **0.4582**), the promotion decision remains `not_promoted`.

Table 5.5: *Current candidate promotion gate*

Gate	Result	Evidence
Holdout PR-AUC ≥ 0.4582	Fail	0.4266
Holdout ROC-AUC ≥ 0.8746	Pass	0.8781
Review precision ≥ 0.30	Pass	0.3820
Calibration non-regression	Pass	Brier decreased
Artifact checksum	Pass	SHA-256 match
Artifact-load smoke	Pass	Manifest verified
Data contract verification	Pass	94 checks ready
Version match	Pass	v2
Git clean	Fail	<code>git_dirty=true</code>

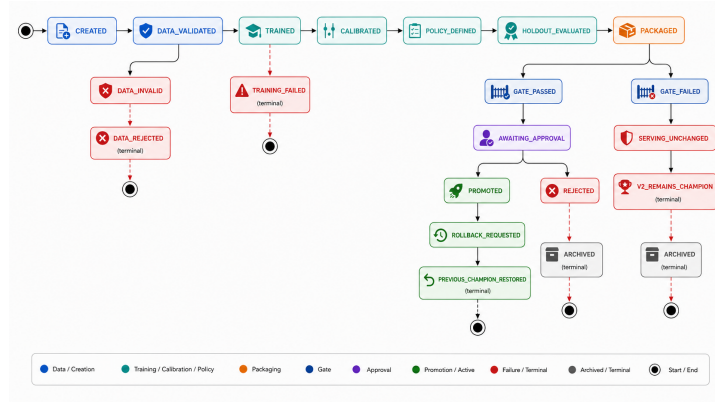


Figure 5.2: *Promotion state machine.*

5.7 Request-time scoring

Real-time scoring maps categoricals via persisted indexers, applies isotonic calibration, and computes TreeSHAP feature attributions [3]. Responses support `full_feature` and `partial_demo` modes.

CHAPTER 6

APPLICATION AND DEPLOYMENT

6.1 Integrated serving architecture

The application operates as an **in-process model-serving modular monolith**. FastAPI imports the scoring package directly, persisting scored transactions in PostgreSQL/SQLite and serving a React dashboard.

6.2 Scoring and review workflow

The backend returns probability, risk score (**0–100**), risk band, decision, and TreeSHAP attributions. Scored transactions and human analyst reviews are stored as distinct database entities.

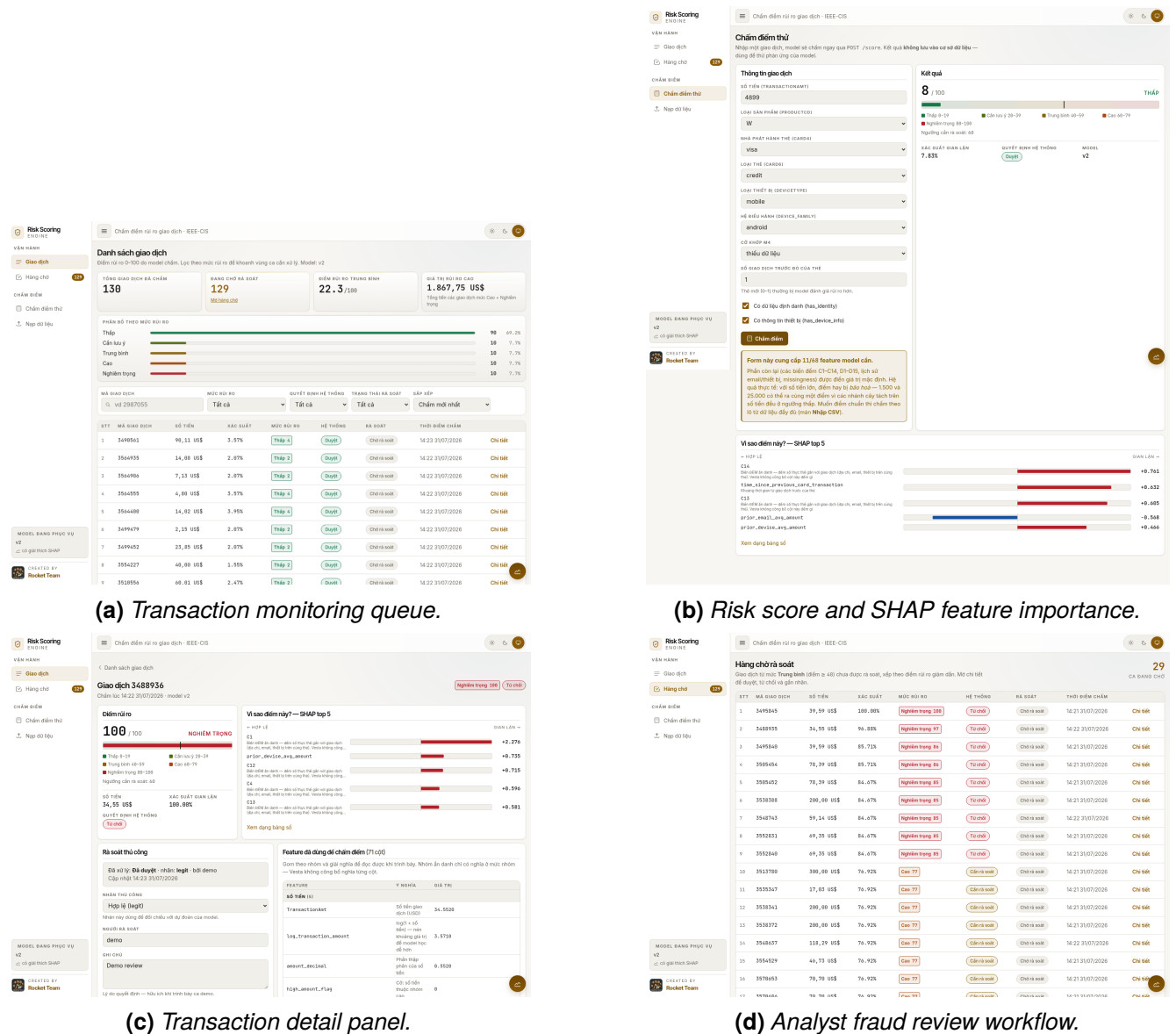


Figure 6.1: Application User Interface: Real-time risk scoring, inspection queue, and analyst review workflow.

6.3 Deployment boundary

Docker Compose orchestrates preprocessing, training, backend, and React UI profiles, mounting processed Parquet artifacts into the backend container.

CHAPTER 7

VALIDATION AND WHOLE-SYSTEM REVIEW

7.1 Evidence status

The verifier passes **94** validation checks with status `ready_for_downstream_training`. Saved artifacts provide empirical evidence for data contracts, baseline comparisons, and candidate holdout evaluation.

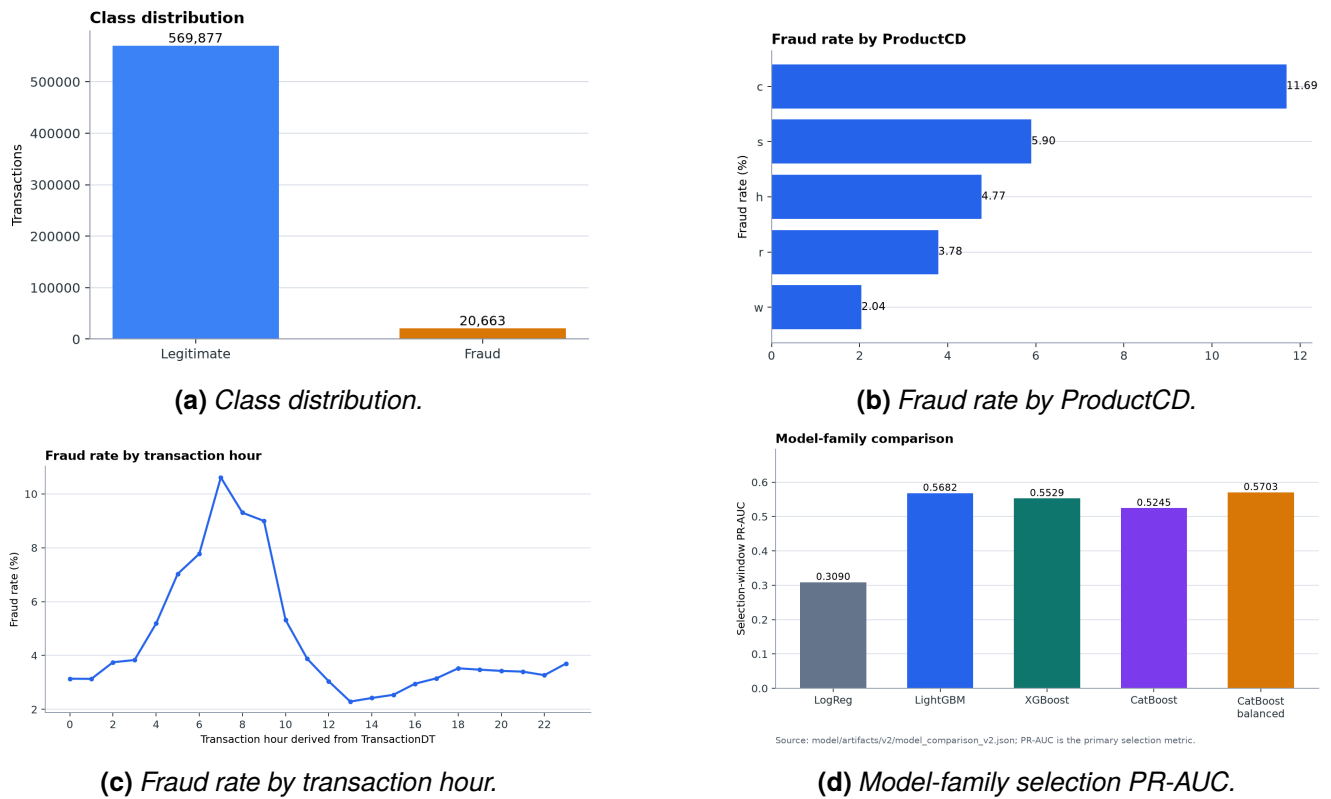


Figure 7.1: Exploratory Data Analysis and Model-Family Selection Evidence.

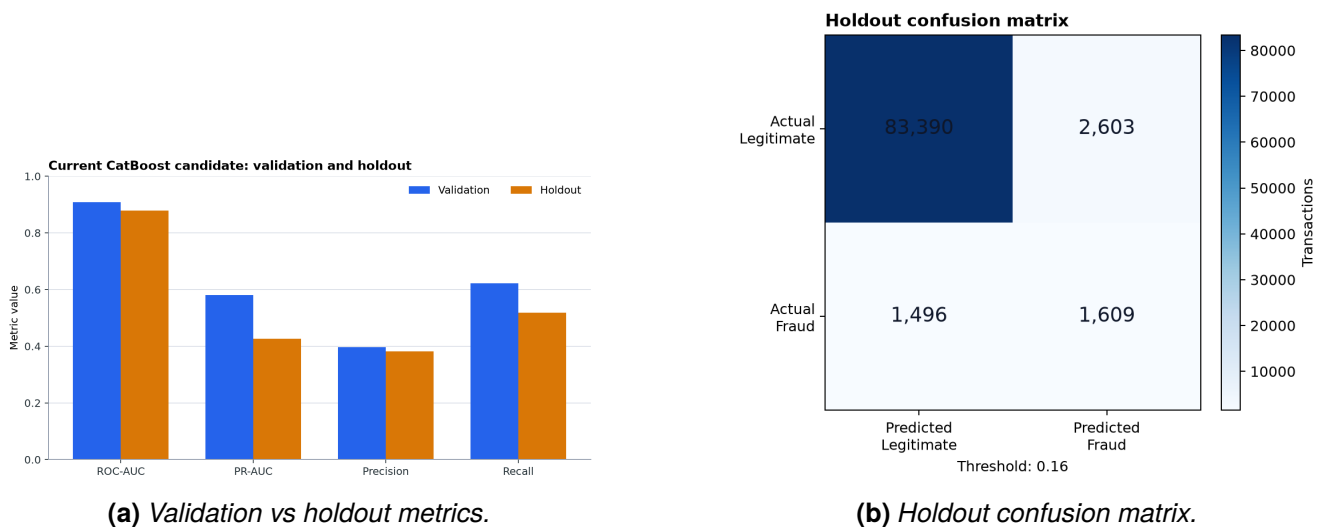


Figure 7.2: Candidate holdout evaluation & confusion matrix.

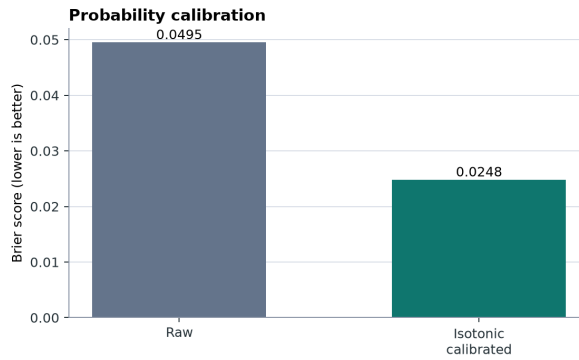


Figure 7.3: Probability calibration: Raw vs isotonic-calibrated Brier score.

7.2 Technical challenges and implemented solutions

Table 7.1 details the technical challenges, solutions, and repository evidence.

Table 7.1: Technical Challenges, Implemented Solutions, and Evidence

Challenge	Implemented solution	Evidence
Transaction–identity sparsity	Transaction-authoritative LEFT JOIN preserving 100% transaction rows with missingness indicators	Preprocessing Join Audit Step
Missing values	Train-only median imputation for numeric; explicit 'missing' category	Feature Contract Specification
Class imbalance	Evaluated original, weighted (weight 3.0), and balanced undersampled splits	Balanced Parquet Artifact
Temporal leakage	Chronological split on TransactionDT (train → val → calib → holdout)	Leakage-free Pipeline Stages
Schema & feature order	Frozen canonical 68-feature vector and machine-checked schema hash gate	Verification Engine Gate
Verifier failure	Automated validation gate outputting ready_for_downstream_training	Verification Report Artifact
Spark-to-pandas boundary	Spark handles distributed data preparation; Python fits single-node matrices	Model Execution Logs
Offline–online feature mismatch	Standardized backend feature loader mapping scoring request vector to schema	Backend Feature Loader

7.3 Consolidated limitations

1. **Reproducibility:** Local MLflow tracking and single-node pandas training.
2. **Policy Alignment:** Runtime backend risk bands decoupled from offline threshold optimization.
3. **Offline-Online Parity:** Real-time feature store for historical aggregates not implemented.
4. **Operational Controls:** Fail-open fallback heuristic and in-memory background jobs.
5. **Security & Governance:** Authentication, role-based authorization, and durable audit logs absent.

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Conclusion

The project demonstrates an end-to-end fraud scoring system separating distributed Spark data preparation from single-node Python model training. CatBoost Balanced achieved validation ROC-AUC **0.9081** and PR-AUC **0.5806** (holdout ROC-AUC **0.8781**, PR-AUC **0.4266**), while candidate status remains `not_promoted`.

8.2 Summary of research question answers

- **RQ1 (Data Integrity):** Leakage-free chronological splits and schema contracts are verified.
- **RQ2 (Classifier Performance):** CatBoost achieved highest PR-AUC; performance degrades gracefully on holdout.
- **RQ3 (Serving Architecture):** In-process Python model package enables Spark-free real-time scoring.
- **RQ4 (System Boundaries):** Architectural limits (offline-online parity, fail-open fallback) bound the demo.

8.3 Future work

To advance the system toward enterprise production deployment, five key engineering and research directions are identified:

1. **Real-Time Feature Store and Online-Offline Parity:** Implement a low-latency online feature store (e.g., Feast or Redis-backed stream processing engine) to calculate streaming aggregations (e.g., rolling 1-hour transaction counts per card) with sub-10ms latency, guaranteeing complete feature parity between offline Spark training and online API inference.
2. **Decoupled Dynamic Policy and Threshold Management:** Separate operational risk decisions (`approve`, `review`, `reject`) from offline model weights. Implement a dynamic policy engine allowing risk officers to update decision thresholds and risk bands in real time without redeploying model artifacts.
3. **Continuous MLOps, Drift Detection, and Auto-Retraining:** Deploy automated monitoring for population stability (PSI) and prediction drift (Kolmogorov-Smirnov tests). Integrate automated CI/CD re-training pipelines triggered when holdout PR-AUC drops below SLA thresholds.
4. **Graph Neural Networks and Temporal Sequence Modeling:** Integrate Graph Neural Networks (GNNs) on transaction-device-entity bipartite graphs to detect coordinated fraud rings, combined with temporal Transformer architectures for sequential pattern analysis.
5. **Actionable Counterfactual Recourse and Analyst UX:** Extend local TreeSHAP explanations with actionable counterfactual recourse, providing analysts with precise quantitative factors necessary to resolve flagged review cases.

BIBLIOGRAPHY

- [1] IEEE Computational Intelligence Society, “leee-cis fraud detection,” 2019.
- [2] Amazon Science, “Fraud dataset benchmark: leee-cis fraud detection,” 2023.
- [3] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

CHAPTER A

API REFERENCE

A.1 Common enums & core fields

```
RiskBand = low | guarded | medium | high | critical  
Decision = approve | review | reject
```

Table A.1: *Core API fields*

Field	Type	Meaning
fraud_probability	float	Model output in $[0, 1]$
risk_score	integer	Probability $\times 100$
risk_band	enum	Backend score band
decision	enum	Policy result (approve/review/reject)
scoring_mode	enum	full_feature or partial_demo
shap_top5	list/null	Local TreeSHAP contributions

A.2 Endpoint summary

Table A.2: *Core REST Endpoints*

Method	Path	Response
GET	/health	Process health status
GET	/meta	Model & schema metadata
POST	/score	Risk score response
GET	/transactions	Paginated transaction queue
GET	/transactions/{id}	Detailed transaction record
POST	/transactions/{id}/review	Analyst review decision update

CHAPTER B

DEPLOYMENT AND FEATURE CONTRACTS

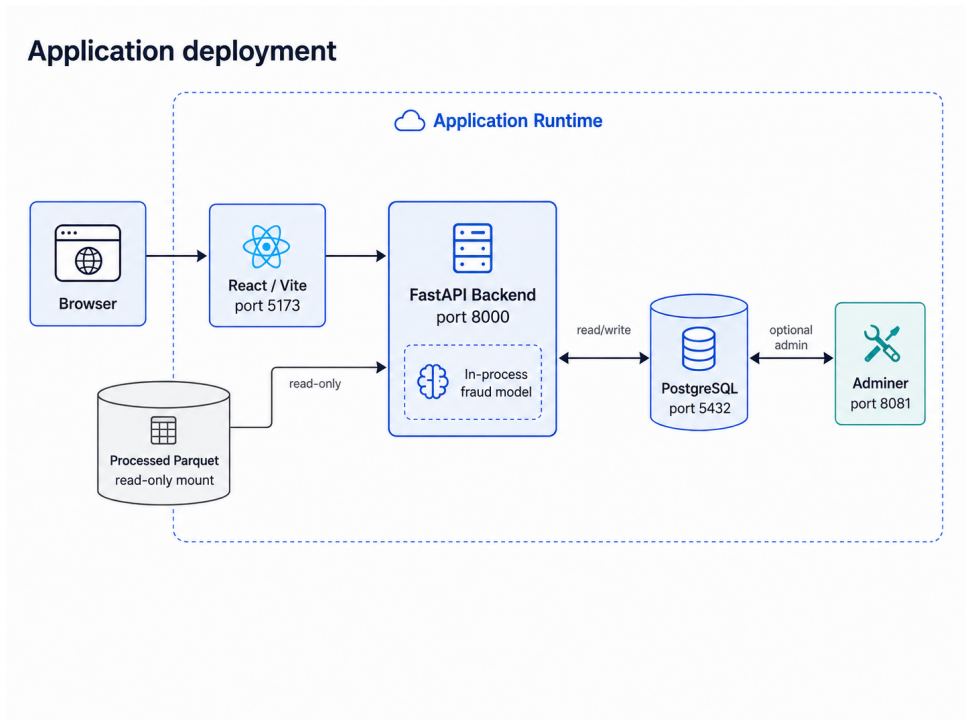


Figure B.1: Default application deployment. Browser, React/Vite, FastAPI backend, PostgreSQL, and in-process fraud model.

B.1 Canonical feature groups

Table B.1: Canonical feature groups

Group	Representative features
Amount	TransactionAmt, log/capped/decimal amount, card-mean ratios
Time	TransactionDT, day, week, hour, day-of-week, age, night flag
Missingness	Selected identity/device missing counts and ratios
Presence	Identity, device, email domain, distance, address flags
Behavior	dist1, dist2, selected C1--C14, selected D1--D15
Categorical	ProductCD, card4, card6, DeviceType, device_family, M4

CHAPTER C

RUNBOOK AND ROLE MAPPING

C.1 Acceptance scenarios

Table C.1: *Acceptance scenarios*

Scenario	Expected evidence
Processed handoff	Verifier passes every schema, row, split, and artifact check
Real model startup	<code>/meta</code> returns expected version with no warning
Full-feature score	<code>scoring_mode=full_feature</code> ; probability bounded
Partial score	<code>partial_demo</code> is visible; required fields enforced
Review update	Status/label persist and queue/statistics invalidate